Универзитет у Београду
Машински факултет

Михајло Катић

Дипломски (B.Sc.) рад

Развој игре за погађање речи коришћењем Qt и MySQL технологија

Београд, 2024. године

Универзитет у Београду
Машински факултет

Основне академске студије:
Информационе технологије у машинству
Предмет:
Објектно оријентисано програмирање

Број регистра: 2018/2020

Михајло Катић

Дипломски (B.Sc.) рад

Развој игре за погађање речи коришћењем Qt и MySQL технологија

Београд, 2024. године

Комисија за преглед и одбрану

др Слободан Радојевић, редовни професор

Андријана Марјановић, асистент

Милош Вучић, асистент

Датум одбране .....

Оцена .....

Предметни наставник

.....
др Слободан Радојевић, редовни професор

Руководилац ОАС-а

.....
др Марко Милош, редовни професор

Овера студентског одсека

Предговор

Главни циљ овог рада је развој игре за погађање речи, уз коришћење Qt C++ окружења, класа, Qt корисничког интерфејса, и базе података повезане преко MySQL-а. У раду ће се детаљно анализирати креирање MySQL базе података и њена интеграција у Qt апликацију, као и дизајн корисничког интерфејса и протокол функционисања саме игре.

Кључне речи

Игра речи, База података, Класа, Кориснички интерфејс, Радни оквир, Метод.

У Београду, 2024. године.

Садржај

1. Увод **5**
1.1. Радно окружење . . . 5
1.1.1. Оперативни систем . . . 5
1.1.2. Програмски Језик . . . 5
1.2. Qt radni okvir . . . 5
1.2.1. Функционалност . . . 5

2. MySql **6**
2.1. Карактеристике . . . 6
2.2. Креирање MySQL базе . . . 7
2.3. Унос података у табелу . . . 7
2.4. Повезивање MySQL базе са Qt радним оквиром . . . 10

3. Игра Речко **11**
3.1. Правила игре . . . 11
3.2. Начин играња . . . 11
3.3. C++ код . . . 15
3.3.1. Start дугме . . . 15
3.3.2. Enter дугме . . . 16

4. Закључак **20**

Садржај Слика

1 Напуњена табела Провера 9

2 Кориснички интерфејс 11

3 Корак 1 12

4 Корак 2 12

5 Корак 3 13

6 Корак 4 13

7 Корак 5 14

8 Корак 6 14

Садржај Кода

1 Инсталација MySQL окружења 7

2 Креирање MySQL базе и табеле 7

3 Попуњавање MySQL табеле 7

4 Унос података у табелу из датотеке 8

5 Инсталација Qt SQL Plugin-a 10

6 Qt SQL Модул 10

7 Повезивање базе у C++ коду 10

8 Притисак на дугме start 15

9 Генерисање тражене речи 15

10 Провера тражене речи 16

11 Погађање тражене речи 17

1. Увод

Развој кратких интернет игара директно кореспондира са напретком савремених рачунарских технологија, укључујући HTML5, C++ и мобилне платформе. Са унапређењем интернет инфраструктуре и растућом доступношћу паметних уређаја, ове игре постале су широко распрострањене, пружајући корисницима интуитивну форму интерактивне забаве и когнитивну стимулацију. Игре као што су "Angry Birds"и "Wordle"примењују напредне алгоритамске моделе и минималистички кориснички интерфејс, чинећи их приступачним широкој публици.
У овом раду се фокусирамо на реплику игре "Wordle"по имену "Речко", користећи Qt у C++ окружењу и MySQL за управљање базом података.

1.1. Радно окружење

1.1.1. Оперативни систем

У овом истраживању, све експериментне и развојне активности спроведене су на Linux оперативном систему, на дистрибуцији Ubuntu. Linux је бесплатан и отворен оперативни систем базиран на Unix-u, који се широко користи у академским и индустријским срединама због своје флексибилности, брзине и поузданости.

1.1.2. Програмски Језик

C++ је објектно-оријентисани програмски језик, што значи да подржава концепте као што су класе и објекти, омогућавајући логично и поновљиво организовање кода. Познат је по својој високој ефикасности и брзини извршавања, као и по ефикасној манипулацији меморијом, што га чини идеалним за системе који захтевају велике перформансе. Поред тога, C++ је мултиплатформски језик и укључује опсежну стандардну библиотеку (STL), која садржи бројне корисне функције и структуре података, значајно олакшавајући развој софтверских апликација.
Главни разлог зашто је C++ популаран у индустрији видео игара је тај што омогућава прецизну манипулацију меморијом, која дозвољава оптимизацију перформанси на најнижем нивоу, неопходном за захтевне игре.

1.2. Qt radni okvir

Qt је C++ оквир за развој софтвера специјализован за израду графичких корисничких интерфејса (GUI).

1.2.1. Функционалност

- **Крос-платформска подршка**: Qt омогућава развој апликација које функционишу на различитим платформама, као што су Windows, macOS, Linux, Android и iOS, без значајнијих измена у коду.

- **Богате графичке могућности**: Пружа алате за израду графичких корисничких интерфејса (GUI), укључујући подршку за адаптивне и респонзивне дизајне.
- **Обликовање и интерактивност**: Са механизмима као што су сигнали и слотови, Qt олакшава управљање догађајима и интеракцијом корисника, што побољшава ефикасност програмирања.
- **Модуларна архитектура**: Састоји се од различитих модула, што омогућава програмерима да користе само оне делове који су им потребни, чиме се смањује величина апликације и побољшавају перформансе.
- **Подршка за мултимедију и мрежу**: Пружа интегрисане алате за рад са мултимедијом, мрежним протоколима и базама података, олакшавајући развој сложених апликација.
- **Интероперабилност**: Може радити са другим језицима и технологијама, укључујући Python (путем PyQt или PySide), што проширује његове могућности.
- **Висока перформанса**: Основан на C++, Qt омогућава оптимизацију и високе перформансе апликација, што је посебно важно за ресурсо-захтевне пројекте.

2. MySql

MySQL је систем за управљање релационим базама података отвореног кода (RDBMS), развијен како би обезбедио ефикасно и поуздано складиштење и управљање подацима. Као један од најпопуларнијих система у својој категорији, MySQL је дизајниран тако да подржава широк спектар апликација и корисничких захтева, од малих веб апликација до сложених и високооптерећених пословних система.

2.1. Карактеристике

- **Поузданост**: MySQL је изграђен са нагласком на стабилност и интегритет података. Подржава ACID (atomicity, consistency, isolation, durability) принципе, што осигурава сигурност података чак и у случају грешака или квара система.
- **Перформансе**: Овај систем је оптимизован за обраду великог броја захтева и података у реалном времену, што га чини погодним за велике апликације као што су веб сервери и електронска трговина.
- **Функционалност**: MySQL се може користити за мање пројекте, али је довољно моћан и за велике корпорације које управљају терабајтима података и милионима корисника.
- **Компатибилност и подршка**: Подржава више платформи, укључујући Windows, Linux и macOS, као и велики број програмских језика (нпр. C++, Java, PHP, Python), чинећи га флексибилним решењем за различите типове пројеката.

- **Отвореност и заједница**: Као пројекат отвореног кода, MySQL има велику корисничку базу и заједницу која активно учествује у његовом развоју и подршци, што значи да постоји обиље ресурса за учење и решавање проблема.

2.2. Креирање MySQL базе

Инсталација MySQL-а за Linux-Ubuntu се најједноставније обавља преко командне линије терминала:

```
sudo apt update

sudo apt install mysql-server

sudo mysql
```

Код 1: Инсталација MySql окружења

Последња команда нас води у MySQL конзолу, одакле уграђеним командама креирамо базу. База за потребе овог пројекта се састоји од две табеле. Састоје се од id-а и једног стринга са 5 карактера. 5 карактера јер се у игри погађа искључиво реч од 5 слова.

```
CREATE DATABASE recko; -- kreiranje baze podataka po imenu recko

USE recko; -- pristup bazi

CREATE TABLE reci ( -- kreiranje tabele
    id INT AUTO_INCREMENT PRIMARY KEY,
    rec VARCHAR(5)
);
```

Код 2: Креирање MySQL базе и табеле

За креирање друге табеле "провера"се понавља горе наведена команда. За пуњење табеле "речи"може се користити уграђена команда:

```
INSERT INTO reci (rec) VALUES ('Stena');
```

Код 3: Попуњавање MySQL табеле

Ова табела представља групу речи, од којих ће случајни генератор изабрати једну реч за погађање у игри. Састоји се од 20 речи због једноставности. Други разлог за ово је тај што друга табела, под називом "провера", служи да провери да ли унета реч постоји у српском језику. Због мале доступности оваквог садржаја, она је непотпуна и садржи речи које нису учестале у модерном језику. Међутим, испуњава све техничке услове и обавља задатак.

2.3. Унос података у табелу

С обзиром да табела има преко 10000 речи, да не бисмо користили "INSERT"команду безброј пута, неопходан је нови механизам који попуњава колоне директно из текстуалне датотеке.

```
#include <mysql_driver.h>
#include <mysql_connection.h>
#include <cppconn/prepared_statement.h>
#include <iostream>
#include <fstream>
#include <vector>
// Funkcija za umetanje podataka u MySQL
void insertDataIntoMySQL(const std::vector<std::string>& data) {
    try {
        // Inicijalizacija MySQL drajvera i veze
        sql::mysql::MySQL_Driver* driver;
        sql::Connection* con;
        driver = sql::mysql::get_mysql_driver_instance();
        con = driver->connect("tcp://127.0.0.1:3306", "username", "password");
        // "miha907", "miha907miha907"

        con->setSchema("recko"); // Izbor baze podataka

        // Priprema SQL upita za umetanje podataka u kolonu "proveri"
        std::unique_ptr<sql::PreparedStatement> pstmt(
            con->prepareStatement("INSERT␣INTO␣provera␣(proveri)␣VALUES␣(?)")
        );

        for (const std::string& line : data) {
            pstmt->setString(1, line);  // Umetanje podataka u kolonu "proveri"
            pstmt->executeUpdate();
        }

        delete con;
    } catch (sql::SQLException& e) {
        std::cerr << "SQLException:␣" << e.what() << std::endl;
    }
}

// Funkcija za citanje teksta iz .txt datoteke
std::vector<std::string> readTxtFile(const std::string& filePath) {
    std::ifstream file(filePath);
    std::vector<std::string> data;
    std::string line;

    while (std::getline(file, line)) {
        data.push_back(line);
    }

    return data;
}

int main(int agrc, char **argv)
{
    std::string filePath = "dodatak.txt"; // ulaz

    std::vector<std::string> data = readTxtFile(filePath);
```

```
    insertDataIntoMySQL(data);

    return 0;
}
```

Код 4: Унос података у табелу из датотеке

У овом C++ коду користе се уграђене функције из горе наведених MySQL библиотека да би се приступило бази и уносиле вредности у жељену табелу из текстуалне датотеке. При компајлирању потребно је додати “-lmysqlcppconn". Излаз је табела:

```
mysql> SELECT * FROM provera;
+-------+---------+
| ID    | proveri |
+-------+---------+
|     1 | abera   |
|     2 | abere   |
|     3 | aberi   |
|     4 | aberu   |
|     5 | avaza   |
|     6 | avazu   |
|     7 | Avaks   |
|     8 | avala   |
|     9 | avale   |
|    10 | avali   |
|    11 | avalu   |
|    12 | avana   |
|    13 | avani   |
|    14 | avans   |
|    15 | avanu   |
|    16 | Avari   |
|    17 | Avelj   |
|    18 | aveti   |
|    19 | aviza   |
|    20 | avion   |
|    21 | AVNOJ   |
|    22 | Avram   |
|    23 | agama   |
|    24 | Agata   |
|    25 | Agate   |
|    26 | agens   |
|    27 | agent   |
|    28 | agina   |
|    29 | agine   |
|    30 | agini   |
|    31 | aginu   |
|    32 | Agnes   |
|    33 | agnec   |
|    34 | agora   |
| 12428 | cuger   |
| 12429 | čukar   |
| 12430 | čunja   |
| 12431 | cupka   |
| 12432 | curim   |
| 12433 | ćurin   |
| 12434 | curio   |
| 12435 | curom   |
| 12436 | cucam   |
| 12437 | cucah   |
| 12438 | cucka   |
| 12439 | cucla   |
| 12440 | cuclu   |
| 12441 | Ruter   |
| 12442 | Balon   |
| 12443 | Pesma   |
| 12444 | Staza   |
| 12445 | Slika   |
| 12446 | Mesto   |
| 12447 | Pismo   |
| 12448 | Grana   |
| 12449 | Rukav   |
| 12450 | Pegla   |
| 12451 | Vreme   |
| 12452 | Tavan   |
| 12453 | Prase   |
| 12454 | Mesto   |
| 12455 | Lampa   |
| 12456 | Lopta   |
| 12457 | Flaša   |
| 12458 | Žvaka   |
| 12459 | Papir   |
| 12460 | Avion   |
| 12461 | ručak   |
+-------+---------+
12461 rows in set (0,01 sec)

mysql>
```

Слика 1: Напуњена табела Провера

2.4. Повезивање MySQL базе са Qt радним оквиром

Након што је база попуњена, потребно ју је повезати са Qt C++ кодом.

- **Инсталација Qt SQL Plugin-a** :
  Додатак за Qt који је неопходан за коришћење MySQL библиотека у Qt оркужењу.

```
sudo apt-get install libqt5sql5-mysql
```

Код 5: Инсталација Qt SQL Plugin-a

- **SQL Модул**:
  Команда за додавање SQL модула у Qt пројекат. Ова команда се уноси у .pro фајл.

```
QT += sql
```

Код 6: Qt SQL Модул

- **Повезивање базе у C++ коду**:

```
#include <QtSql/QSqlDatabase>
#include <QtSql/QSqlError>
#include <QtSql/QSqlQuery>
#include <QDebug>

int main(int argc, char *argv[])
{
QApplication a(argc, argv);
MainWindow w;
w.show();

// Kreira QSqlDatabase objekat i postavlja drajver
QSqlDatabase db = QSqlDatabase::addDatabase("QMYSQL");

//parametri za konekciju sa bazom
db.setHostName("localhost");       // host
db.setDatabaseName("recko"); // naziv baze
db.setUserName("miha907");   // username
db.setPassword("miha907miha907");   // password

// Otvori konekciju sa bazom podataka
if (!db.open())
{
    qDebug() << "Baza nije povezana.";
    qDebug() << db.lastError().text();
    return -1;
};

qDebug() << "Baza je povezana!";

return a.exec();
```

```
};
```

Код 7: Повезивање базе у C++ коду

QSqlDatabase је класа библиотеке < QtSql / QSqlDatabase > која повезује базу. Њене уграђене функције подешавају параметре за приступ специфичној бази. Ако је база успешно повезана, на излазу се штампа порука "Baza je povezana!".

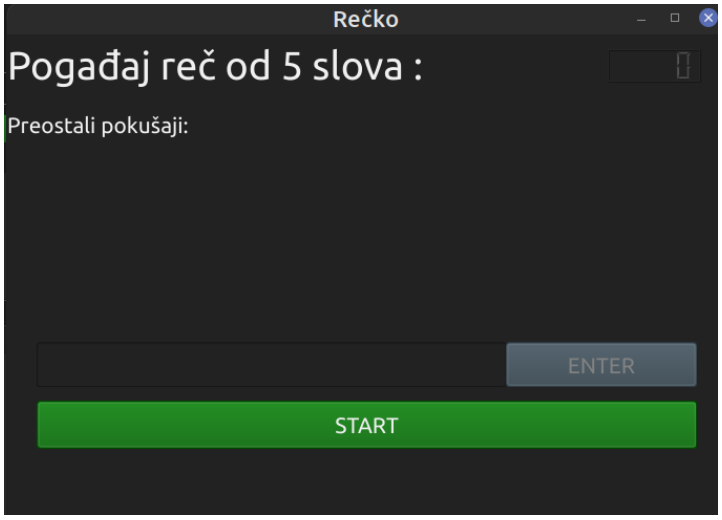
3. Игра Речко

3.1. Правила игре

Притиском на старт дугме, програм покреће тајмер и генерише српску реч од 5 слова , коју треба погодити из 6 покушаја. Након сваке унете речи, игра за свако слово враћа један од 3 излаза: сиво слово - слово није садржано у траженој речи, жуто слово - слово је садржано у траженој речи, али није на добром месту, и зелено слово - слово је садржано у траженој речи и на добром је месту. Ако је реч погођена, тајмер се зауставља и мери укупно време игре, исписује се тражена реч и порука "Победа!". У случају да се након 6 покушаја не погоди реч, тајмер се зауставља и мери укупно време игре, исписује се тражена реч и порука "Пораз!".
Игра није осетљива на мала и велика слова, али јесте учитана српска тастатура (ш, ђ, ћ, ж, ч).

3.2. Начин играња

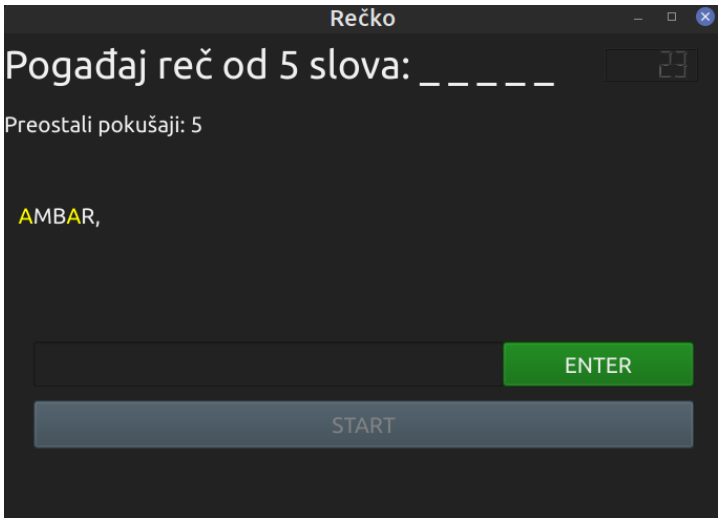
Након компајлирања и покретања, аутоматски се позива кориснички интерфејс:



Слика 2: Кориснички интерфејс

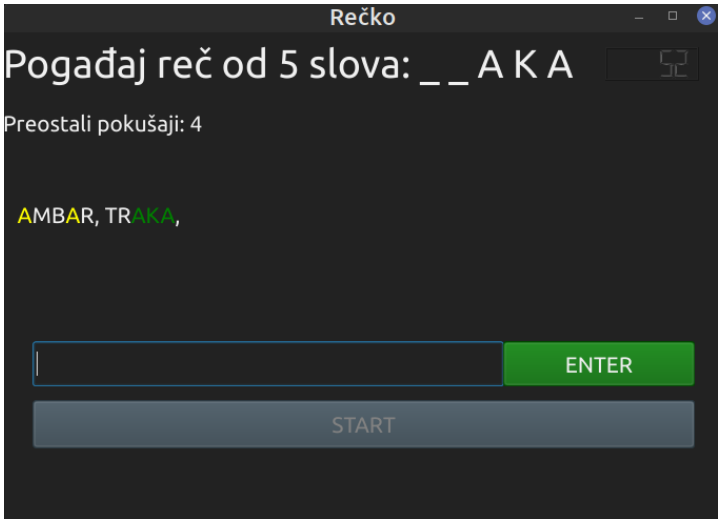
Притиском на "START"дугме, отвара се дугме "ENTER"и поље за унос речи. У овом при-

меру, изабрана реч је "Žvaka", тако да први покушај, реч "Ambar"садржи два слова "a", али ни једно није на месту. Трака са траженом речју остаје празна и знамо да постоје два слова "a"која нису на 1. и 4. месту.



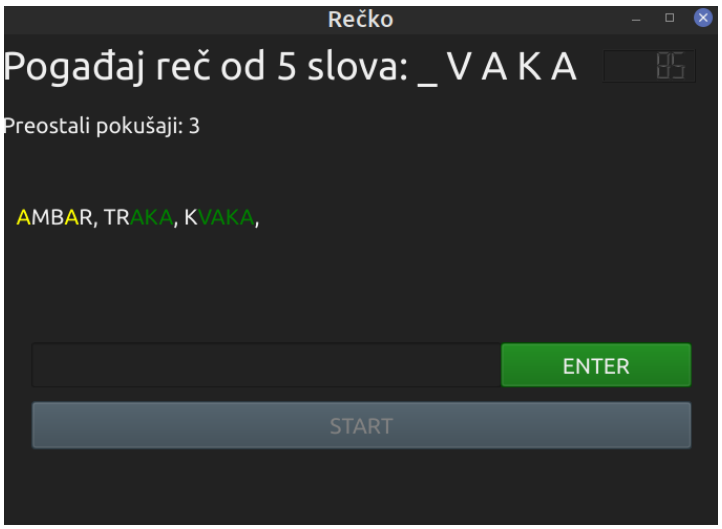
Слика 3: Корак 1

Следећи покушај, реч "Traka"је одличан јер садржи само нова, неопробана, слова и два слова "a"која су на новим местима. Погођена слова постају зелена и откривају се на исецканој траци тражене речи.



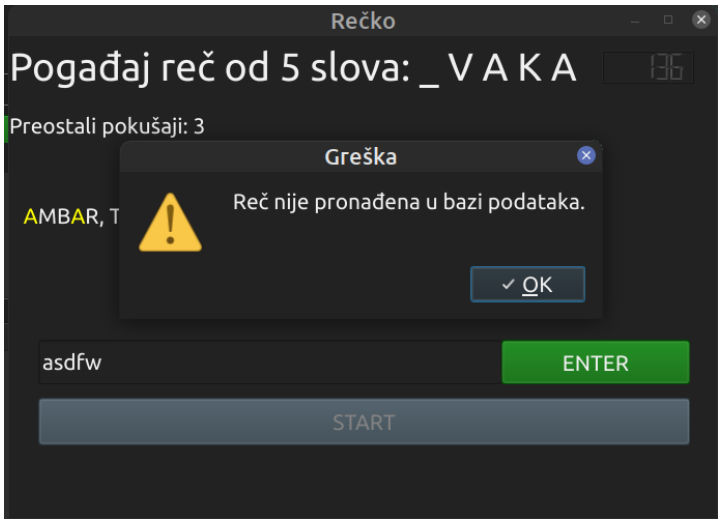
Слика 4: Корак 2

Реч "Kvaka"је следећи покушај, након ње остајемо са 3 преостала покушаја и једним словом које недостаје.



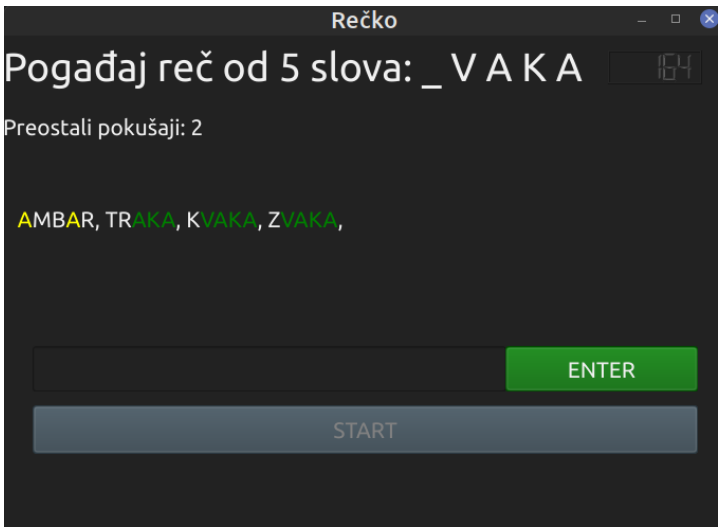
Слика 5: Корак 3

Тест рада базе података и табеле "provera". Не дозвољава унос речи која не припада колони "proveri"и избацује грешку.



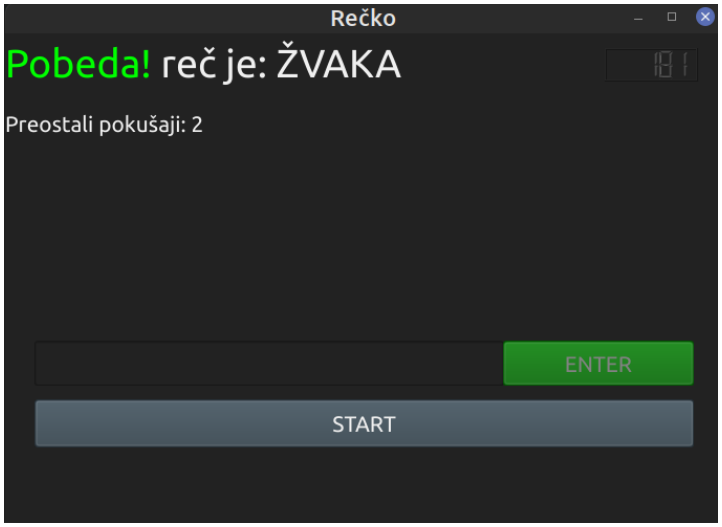
Слика 6: Корак 4

Провера сензитивности на српску азбуку. Омогућен је унос, али не препознаје реч као исправну.



Слика 7: Корак 5

Коначно, унос је реч "Žvaka", тајмер се зауставља, исписује се тражена реч и порука "Pobeda". Поновно покретање игре, са новом генерисаном речју је могуће извршити притиском на дугме "start", где се цео циклус понавља.



Слика 8: Корак 6

3.3. C++ код

Два дугмета, "Start"и "Enter", или како се у Qt оквиру називају "слотови", покрећу све функције овог пројекта.

3.3.1. Start дугме

```
void MainWindow::on_startButton_clicked()
{
    generateSecretWord();

    ui->wordLabel->setText("Pogađaj␣reč␣od␣5␣slova␣:␣" + QString(guessedWord.
        length(), '_'));

    ui->startButton->setEnabled(false);
    ui->startButton->setStyleSheet("background-color:␣#36454F");
    ui->guessButton->setEnabled(true);
    ui->guessButton->setStyleSheet("background-color:␣#006400");
    ui->guessLineEdit->setEnabled(true);

    number_of_guesses = 6;

    ui->attemptsLabel->setText("Preostali␣pokušaji:␣" + QString::number(
        number_of_guesses));

    secondsElapsed = 0;

    if (timer->isActive()) {
            timer->stop();
        }

        timer->start(1000);
}
```

Код 8: Притисак на дугме start

Слот функција, која нема повратну вредност (void) позива се као приватна функција класе "MainWindow". Поставља елементе корисничког интерфејса на жељене вредности: гаси старт дугме, активира ентер дугме и линију за унос. Поставља број покушаја и активира тајмер. Активира функцију за генерисање тражене речи.

- **generateSecretWord**:

```
void MainWindow::generateSecretWord()
{
    QSqlQuery query;

    if (!query.exec("SELECT␣*␣FROM␣reci"))
    {
        QMessageBox::critical(this, "Greška", "Neuspešno␣pristupanje␣bazi:␣" +
            query.lastError().text());
```

```
        return;
    }

    QStringList words;
    while (query.next())
    {
        QString word = query.value(1).toString();
        words.append(word);
    }

    if (words.isEmpty())
    {
        QMessageBox::critical(this, "Greška", "Baza je prazna.");
        return;
    }

    QRandomGenerator randomGenerator(QDateTime::currentMSecsSinceEpoch() /
        1000);
    int index = randomGenerator.bounded(0, words.size());
    secretWord = words.at(index);
    guessedWord = QString(secretWord.length(), '_');
}
```

Код 9: Генерисање тражене речи

Потребно је приступити бази да би из ње генерисали реч. За то је потребан SQL објекат уграђене класе. Када је упит за одговарајућу табелу враћен, while петља пролази кроз њега, учитава вредност у стринг који се убацује у повезану листу. Генератор случајних бројева се позива преко уграђене класе, а за семе узима време у милисекундама од Unix епохе (01.01.1970) и дели га са 1000. Бира реч из опсега и поставља је као тражену реч.

3.3.2. Enter дугме

Поред елемената корисничког интерфејса, проверава да ли унета реч има 5 слова, на сваку успешно унету реч смањује број покушаја за 1, ако се број покушаја спусти на 0, гаси игру и исписује поруку. Позива две методе: isGuessCorrect и checkStringExists.

- **checkStringExists**:

```
    bool MainWindow::checkStringExists(const QString& inputString)
{
    QSqlQuery query;
    query.prepare("SELECT COUNT(*) FROM provera WHERE proveri = ?");
    query.addBindValue(inputString);

    if (!query.exec())
    {
        QMessageBox::critical(this, "Error", "Failed to execute query: " +
            query.lastError().text());
        return false;
    }
```

```

    if (query.next())
    {
        int count = query.value(0).toInt();
        return count > 0;
    }

    return false;
}
```

Код 10: Провера тражене речи

Ова метода проверава да ли је унети стринг део табеле базе података. Креира SQL објекат који проверава колико редова у табели "provera"има вредност из "proveri"која је једнака вредности улазног стринга. Вредност је 0 или 1, пошто се стрингови не понављају.

- **isGuessCorrect**:

```
    bool MainWindow::isGuessCorrect(const QString &guess)
{
    QString uppercaseGuess = guess.toUpper();
    QString lowercaseGuess = guess.toLower();
    QString lowercaseSecretWord = secretWord.toLower();

    if (guessedWord.isEmpty())
    {
        guessedWord = QString(secretWord.length(), '_');
    }

    QMap<QChar, int> secretWordFrequency;
    QMap<QChar, int> guessFrequency;
    QMap<QChar, int> correctGuessFrequency;

    for (int i = 0; i < lowercaseSecretWord.length(); ++i)
    {
        QChar charSecret = lowercaseSecretWord.at(i);
        secretWordFrequency[charSecret]++;
    }

    for (int i = 0; i < lowercaseGuess.length(); ++i)
    {
        QChar charGuess = lowercaseGuess.at(i);
        guessFrequency[charGuess]++;
    }

    QStringList feedback(secretWord.length(), QString());
    QMap<QChar, int> usedInGuess;

    for (int i = 0; i < secretWord.length(); ++i)
    {
        QChar guessChar = lowercaseGuess.at(i);
        QChar secretChar = lowercaseSecretWord.at(i);
```

```
        if (guessChar == secretChar)
        {
            feedback[i] = QString("<font color=\"green\">%1</font>").arg(uppercaseGuess.at(i));
            correctGuessFrequency[guessChar]++;
            usedInGuess[guessChar]++;
            secretWordFrequency[secretChar]--;

            guessedWord[i] = uppercaseGuess.at(i);
        }
    }

    for (int i = 0; i < secretWord.length(); ++i)
    {
        if (feedback[i].isEmpty())
        {
            QChar guessChar = lowercaseGuess.at(i);
            if (secretWordFrequency.contains(guessChar) && secretWordFrequency[guessChar] > 0)
            {
                int availableInstances = secretWordFrequency[guessChar] + correctGuessFrequency[guessChar] - usedInGuess.value(guessChar, 0);
                if (availableInstances > 0)
                {
                    feedback[i] = QString("<font color=\"yellow\">%1</font>").arg(uppercaseGuess.at(i));
                    usedInGuess[guessChar]++;
                } else {
                    feedback[i] = QString("%1").arg(uppercaseGuess.at(i));
                }
            } else {
                feedback[i] = QString("%1").arg(uppercaseGuess.at(i));
            }
        }
    }

    QString feedbackHtml = feedback.join("");
    ui->feedbackLabel->setText(ui->feedbackLabel->text() + QString(" ") + feedbackHtml + QString(", "));
    QString spacedGuessedWord = guessedWord.split("").join(" ").trimmed();
    ui->wordLabel->setText("Pogađaj reč: " + spacedGuessedWord);

    if (lowercaseGuess == lowercaseSecretWord)
    {
        ui->wordLabel->setText("<font color=\"lime\">Pobeda!</font> reč je: " + secretWord.toUpper());
        timer->stop();
        return true;
    } else
    {
```

```
        ui->wordLabel->setText("Pogađaj reč od 5 slova: " +
            spacedGuessedWord.toUpper());
        return false;
    }
}
}
```

Код 11: Погађање тражене речи

Прво, улазна и тражена реч се конвертују у мала слова, ради поређења. Постављају се Мапе, структура података за управљање елементима по систему кључ-вредност, које се користе за бројење учесталости карактера у унетој и траженој речи. Прва петља проверава слова на тачној позицији, ако је то случај, умањује учесталост тог слова и ажурира слово у траженој речи. Друга петља проверава слова на погрешној позицији, а припадају траженој речи. Користи мапе како би означила тачан број слова. Коначно, провера да ли је унета реч једнака траженој, ако јесте, исписује поруку за победу, зауставља тајмер и исписује тражену реч.

4. Закључак

Овај дипломски рад показао је како се савремени програмски алати, као што су C++ и Qt, могу успешно применити у креирању интерактивних апликација које комбинују напредне алгоритме и кориснички интерфејс прилагођен различитим уређајима. Интеграција са MySQL базом података омогућила је ефикасно управљање речима које су основа игре, чиме се демонстрира флексибилност и моћ овог система за управљање базама података.

Кроз овај пројекат, истражене су технике генерисања случајних података, манипулације стринговима, као и концепти везани за објектно-оријентисано програмирање, који дају основу за даљи развитак и проширење овог пројекта.
Циљеви овог рада су остварени, али увек постоји места за побољшање. Могући правци унапређења су: проширење базе података са речима, унапређење корисничког интерфејса, делокализација базе података и додавање нових режима игре.

Литература

[1] https://www.quora.com/Why-is-C++-so-popular-in-game-programming

[2] https://www.elektronika.ftn.uns.ac.rs/racunarska-elektronika/wp-content/uploads/sites/21/2018/03/V10.pdf

[3] https://dev.mysql.com/doc/

[4] https://wiki.qt.io/About_Qt